Marzo 2026

# Manual TÉCNICO

Práctica Única
Lenguajes Formales y de Programación

Allan Acabal

Marzo 2026

# Manual TÉCNICO

**Práctica Única**
**Lenguajes Formales y de Programación**

Allan Acabal

# Introducción

El presente manual técnico describe la arquitectura, componentes y funcionamiento interno del “Sistema de Análisis Académico”, una aplicación desarrollada en C++ que procesa información académica contenida en archivos de texto con formato .lfp.

El sistema permite cargar datos de estudiantes, cursos y notas, realizar cálculos estadísticos avanzados y generar reportes en formato HTML.

Este documento está dirigido a desarrolladores, mantenedores y cualquier persona interesada en comprender los detalles de implementación, las estructuras de datos empleadas, los algoritmos utilizados y las decisiones técnicas tomadas durante el desarrollo.

# Requerimientos Técnicos:

## Entorno de Desarrollo:

- **Lenguaje:** C++ (estándar C++11 o superior)
- **Compilador:** Compatible con MSVC (Visual Studio), g++ o clang++
- **Sistema operativo:** Windows, Linux o macOS (código multiplataforma)
- **Herramientas:** Visual Studio (o cualquier IDE con soporte para C++)

## Librerías Utilizadas (STL):

El proyecto utiliza exclusivamente la biblioteca estándar de C++ (STL) para garantizar portabilidad y cumplir con las restricciones de la práctica. Las librerías empleadas son:

- <**iostream**>: Entrada/salida por consola.
- <**fstream**>: Manejo de archivos (lectura/escritura).
- <**sstream**>: Separación de cadenas (stringstream).
- <**string**>: Manejo de cadenas de texto.
- <**algorithm**>: Algoritmos como sort, max_element, min_element.
- <**cmath**>: Funciones matemáticas (sqrt).
- <**iomanip**>: Formato de salida (setprecision, fixed).
- <**map**>: Utilizado en el reporte de análisis por carrera para la distribución por semestre.

No se emplean librerías externas ni frameworks adicionales.

## Estructura de Datos:

Las estructuras de datos representan las entidades del dominio y se definen en el archivo Estructuras.h. Se optó por struct para simplicidad, ya que todos los miembros son públicos.

```
struct Estudiante {
    int carnet;
    std::string nombre;
    std::string apellido;
    std::string carrera;
    int semestre;
};

struct Curso {
    int codigo;
    std::string nombre;
    int creditos;
    int semestre;
    std::string carrera;
};

struct Nota {
    int carnet;
    int codigo_curso;
    double nota;
    std::string ciclo;
    int anio;
};
```

Además, se definen constantes para los tamaños máximos de los arreglos estáticos:

```
const int MAX_ESTUDIANTES = 100;
const int MAX_CURSOS = 50;
const int MAX_NOTAS = 500;
```

## Funciones Principales:

Las funciones se organizan en dos módulos: Funciones (operaciones básicas y estadísticas) y Reportes (generación de HTML):

## Módulo Funciones (Funciones.cpp):

- **cargarEstudiantes, cargarCursos, cargarNotas:** Leen los archivos .lfp, separan campos y almacenan en arreglos. Devuelven el número de elementos cargados.
- **notasDeEstudiante, notasDeCurso:** Filtran el arreglo de notas por carnet o código de curso, devolviendo un arreglo con las coincidencias.
- **buscarIndiceEstudiante, buscarIndiceCurso:** Buscan un estudiante o curso por su identificador y devuelven su índice en el arreglo (o -1 si no existe).
- **media, mediana, desviacionEstandar, percentil:** Funciones estadísticas que operan sobre arreglos de double.
- **extraerValoresNotas:** Convierte un arreglo de Nota en un arreglo de double con las calificaciones.

## Módulo Reportes (Reportes.cpp):

- **reporteEstadisticasPorCurso:** Genera un HTML con estadísticas de cada curso.
- **reporteRendimientoPorEstudiante:** Muestra el rendimiento individual de cada estudiante.
- **reporteTop10Estudiantes:** Lista los 10 mejores promedios.
- **reporteCursosMayorReprobacion:** Ordena cursos por porcentaje de reprobados.
- **reporteAnalisisPorCarrera:** Presenta análisis agrupado por carrera.

# Pseudocódigo

## Lectura de Archivos:

## Carga de Estudiantes:

```
Función cargarEstudiantes(arreglo estudiantes[], max)
    archivo = abrir("estudiantes.lfp")
    si no se pudo abrir:
        mostrar error
        retornar 0
    contador = 0
    mientras haya líneas y contador < max:
        leer línea
        si línea no está vacía:
            crear stringstream con la línea
            extraer campo1 (carnet) con getline(, ',')
            convertir a entero con stoi
            extraer campo2 (nombre)
            extraer campo3 (apellido)
            extraer campo4 (carrera)
            extraer campo5 (semestre) y convertir a entero
            asignar valores a estudiantes[contador]
            contador++
        fin si
    fin mientras
    cerrar archivo
    retornar contador
fin función
```

## Carga de Cursos

```
Función cargarCursos(arreglo cursos[], max)
    archivo = abrir("cursos.lfp")
    si no se pudo abrir:
        mostrar error
        retornar 0
    contador = 0
    mientras haya líneas y contador < max:
        leer línea
        si línea no está vacía:
            crear stringstream con la línea
            extraer campo1 (código) con getline(, ',')
            convertir a entero con stoi
            extraer campo2 (nombre)
            extraer campo3 (créditos) y convertir a entero
            extraer campo4 (semestre) y convertir a entero
            extraer campo5 (carrera)
            asignar valores a cursos[contador] (código, nombre, créditos, semestre,
carrera)
            contador++
        fin si
    fin mientras
    cerrar archivo
    retornar contador
fin función
```

## Carga de Notas:

```
Función cargarNotas(arreglo notas[], max)
    archivo = abrir("notas.lfp")
    si no se pudo abrir:
        mostrar error
        retornar 0
    contador = 0
    mientras haya líneas y contador < max:
        leer línea
        si línea no está vacía:
            crear stringstream con la línea
            extraer campo1 (carnet) con getline(, ',')
            convertir a entero con stoi
            extraer campo2 (código_curso) y convertir a entero
            extraer campo3 (nota) y convertir a double con stod
            extraer campo4 (ciclo)
            extraer campo5 (año) y convertir a entero
            asignar valores a notas[contador] (carnet, código_curso, nota, ciclo, año)
            contador++
        fin si
    fin mientras
    cerrar archivo
    retornar contador
fin función
```

## Cálculos Estadísticos (Reporte 1: Estadísticas por Curso)

```
Función reporteEstadisticasPorCurso(cursos[], totalCursos, notas[], totalNotas)
    crear archivo HTML
    escribir cabecera y estilos
    para cada curso en cursos:
        notasCurso = notasDeCurso(curso.codigo, notas, totalNotas)
        si notasCurso no vacío:
            valores = extraerValoresNotas(notasCurso)
            promedio = media(valores)
            max = max_element(valores)
            min = min_element(valores)
            desv = desviacionEstandar(valores)
            med = mediana(valores)
            escribir fila en tabla HTML con estos datos
        fin si
    fin para
    cerrar archivo
fin función
```

# Menú Principal

Aplicando manejo de excepciones se interpreta de la siguiente forma:

```
Inicio
    declarar arreglos estudiantes, cursos, notas y sus contadores
    repetir:
        mostrar menú
        leer opción
        si opción entre 1 y 3:
            según opción:
                1: contadorEst = cargarEstudiantes(estudiantes, MAX_EST)
                2: contadorCur = cargarCursos(cursos, MAX_CUR)
                3: contadorNot = cargarNotas(notas, MAX_NOT)
            mostrar cantidad cargada
        sino si opción entre 4 y 8:
            verificar que los datos necesarios estén cargados
            si no: mostrar mensaje y continuar
            según opción:
                4: reporteEstadisticasPorCurso(cursos, contadorCur, notas, contadorNot)
                5: reporteRendimientoPorEstudiante(estudiantes, contadorEst, cursos, contadorCur, notas, contadorNot)
                6: reporteTop10Estudiantes(estudiantes, contadorEst, notas, contadorNot)
                7: reporteCursosMayorReprobacion(cursos, contadorCur, notas, contadorNot)
                8: reporteAnalisisPorCarrera(estudiantes, contadorEst, cursos, contadorCur, notas, contadorNot)
            mostrar confirmación
        sino si opción == 9:
            salir
        sino:
            mostrar "Opción inválida"
    mientras opción != 9
Fin
```

# Formato de Archivos de Entrada

Los archivos de entrada son de texto plano con extensión.lfp. Cada línea representa un registro y los campos están separados por comas (,). No hay líneas de encabezado.

**estudiantes.lfp:**

**Formato:** carnet,nombre,apellido,carrera,semestre

estudiantes.lfp | cursos.lfp | notas.lfp

Archivo Editar Ver

```
202016789,Pedro,Ramírez,Sistemas,6
202017890,Luisa,Hernández,Industrial,4
202018901,Diego,Castillo,Civil,8
202019012,Sofía,Morales,Sistemas,3
202020123,Andrés,Reyes,Industrial,5
202021234,Valeria,Ortiz,Civil,6
202022345,José,Chávez,Sistemas,7
202023456,Camila,Navarro,Industrial,2
202024001,María,Gómez,Sistemas,2
202024002,Carlos,López,Industrial,3
202024003,Ana,Martínez,Civil,4
202024004,Jorge,Pérez,Sistemas,1
202024005,Daniela,Ruiz,Industrial,6
202024006,Miguel,Torres,Civil,5
202024007,Paula,Jiménez,Sistemas,4
202024008,Roberto,Sánchez,Industrial,7
```

**cursos.flp:**

**Formato:** codigo,nombre,creditos,semestre,carrera

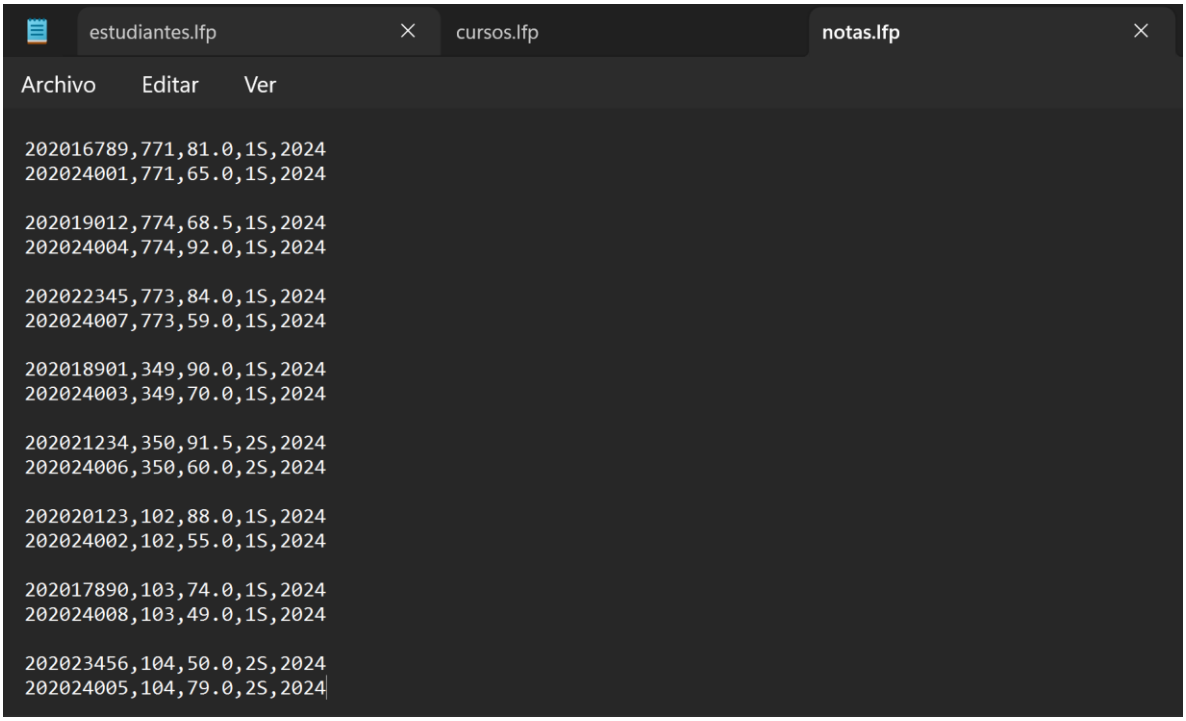
estudiantes.lfp | cursos.lfp | notas.lfp

Archivo Editar Ver

```
771,Estructuras de Datos,5,3,Sistemas
773,Sistemas Operativos 1,5,4,Sistemas
774,Bases de Datos 1,5,4,Sistemas
349,Mecánica de Fluidos,6,6,Civil
350,Análisis Estructural 1,6,6,Civil
102,Cálculo II,4,2,Industrial
103,Física I,4,1,Industrial
104,Estadística 1,4,3,Industrial
```

**notas.lfp**

**Formato:** carnet,codigo_curso,nota,ciclo,año




```
202016789,771,81.0,1S,2024
202024001,771,65.0,1S,2024

202019012,774,68.5,1S,2024
202024004,774,92.0,1S,2024

202022345,773,84.0,1S,2024
202024007,773,59.0,1S,2024

202018901,349,90.0,1S,2024
202024003,349,70.0,1S,2024

202021234,350,91.5,2S,2024
202024006,350,60.0,2S,2024

202020123,102,88.0,1S,2024
202024002,102,55.0,1S,2024

202017890,103,74.0,1S,2024
202024008,103,49.0,1S,2024

202023456,104,50.0,2S,2024
202024005,104,79.0,2S,2024
```

## Archivos de Salida HTML

Todos los archivos incluyen:

- Declaración <!DOCTYPE html> y codificación UTF-8.
- Estilos CSS incrustados para mejorar la presentación de tablas.
- Tablas con bordes, colores alternos y cabeceras destacadas.
- Datos numéricos formateados con dos decimales (excepto conteos enteros).

| Reporte | Archivo Generado |
|---|---|
| **Estadísticas por Curso** | reporte_estadisticas_por_curso.html |
| **Rendimiento por Estudiante** | reporte_rendimiento_estudiante.html |
| **Top 10 Mejores Estudiantes** | reporte_top10_estudiantes.html |
| **Cursos con Mayor Reprobación** | reporte_cursos_reprobacion.html |
| **Análisis por Carrera** | reporte_analisis_carrera.html |

## Observaciones del Desarrollo

- **Elección de arreglos estáticos:** Se optó por arreglos en lugar de vector para simplificar la gestión de memoria y evitar dependencias de la STL que pudieran ser cuestionadas en un contexto educativo. Sin embargo, se definieron tamaños máximos conservadores para cubrir la mayoría de los casos.
- **Manejo de errores:** Aunque no se implementaron excepciones con try-catch, se incluyeron validaciones básicas (existencia de archivos, datos cargados antes de reportes) para guiar al usuario.
- **Portabilidad:** El código es compatible con cualquier compilador C++11, lo que permite ejecutarlo en distintos sistemas operativos sin modificaciones.
- **Extensibilidad:** Las funciones están diseñadas de manera modular, facilitando la adición de nuevos reportes o cálculos estadísticos en el futuro.
- **Limitaciones conocidas:** El programa no soporta archivos con más registros que los límites definidos; en tal caso, se trunca la lectura y se muestra una advertencia. Además, no se validan formatos incorrectos (por ejemplo, letras donde se esperan números), asumiendo que los archivos de entrada cumplen la especificación.